

✓ LangGraph Quiz — Complete Answer Key

THEORETICAL ANSWERS

1. What is LangGraph and who authored it?

LangGraph is an **open-source library** authored by **LangChain Inc.** It is a **low-level orchestration framework and runtime** for building, managing, and deploying **long-running, stateful agents**. It models AI workflows as a **graph** instead of a chain.

2. What are the four characteristics that LangGraph was designed to enable in cognitive architectures?

LangGraph was designed to enable:

1. **Multiactor** — multiple actors (LLMs, tools, agents) working together
 2. **Multistep** — workflows broken into many steps
 3. **Stateful** — maintains state across steps
 4. **Cognitive architectures** — designs that mimic structured "thinking systems"
-

3. Why does LangGraph model AI workflows as a graph instead of a chain? Give one advantage.

A **chain** is strictly linear ($A \rightarrow B \rightarrow C$). A **graph** allows **branching, looping, parallel execution, and conditional routing**. **Advantage:** It supports **complex workflows** like agent loops (assistant $\leftrightarrow$ tools), parallel branches, and conditional decisions — none of which are possible in a simple chain.

4. Name the three projects/libraries that inspired LangGraph.

- **Pregel** $\rightarrow$ inspired the underlying graph engine
 - **Apache Beam** $\rightarrow$ also inspired the engine
 - **NetworkX** $\rightarrow$ inspired the **public interface** (how developers interact with it)
-

5. Can you use LangGraph without LangChain? Why or why not?

Yes. LangGraph is built by LangChain Inc., **but it can be used without LangChain**. LangChain just provides **prebuilt agent architectures** for common LLM and tool-calling loops, which makes

things easier — but it isn't required.

6. What is meant by **multiactor**? List 4 examples of "actors."

Multiactor means multiple specialized components working together (a "team of specialists"). An LLM prompt is much more powerful when paired with related artifacts:

1. **Prompts**
 2. **Tools**
 3. **Agents**
 4. **MCP Servers**
-

7. List the three things an actor coordination layer must manage.

1. **Define the actors involved** → these become **nodes** in a graph
 2. **How they hand off work to each other** → these are the **edges** in the graph
 3. **Schedule execution** of each actor
-

8. What three benefits do you get from a single central state?

1. **Snapshot and store** the central state during/after each computation
 2. **Pause and resume** execution, making **error recovery** easy
 3. **Implement human-in-the-loop controls**
-

9. What does DAG stand for, and why is LangGraph called a DAG?

DAG = Directed Acyclic Graph.

- **Directed** → edges have direction ($A \rightarrow B$, not just $A - B$)
- **Acyclic** → no infinite loops by default

LangGraph models workflows as a DAG because each node passes work in a specific direction toward the END node.

10. Define the three building blocks of a graph.

- **State** → the data received from outside the application, modified and produced as the

application runs.

- **Nodes** → each step to be taken. Nodes are simply **Python/JS functions (tools)** that receive the current state as input and return an update to it.
 - **Edges** → connections between nodes that determine the path. They can be **fixed** or **conditional**.
-

11. Difference between fixed edge and conditional edge?

- **Fixed (Normal) Edge:** Always goes from Point A → Point B, no exceptions.
 - **Conditional Edge:** Uses a Python function to **decide which path to take** based on the current state.
-

12. Why is a conditional edge "not really an edge"?

Because it's actually a **Python function that decides which node to visit next**, not a static line connecting two fixed nodes. It evaluates logic and returns the **name of the next node**.

13. What are the START and END nodes?

Special nodes that indicate the **beginning** (START) and **termination** (END) of the graph. The graph's flow always begins at START and finishes at END.

14. What is a reducer and why is it crucial for parallelism?

A **reducer** defines how **simultaneous/competing updates from multiple actors** are combined deterministically.

It's crucial for parallelism because if two parallel branches both update the same key (e.g., `messages`), there's ambiguity — should the final state contain only the first update? Both? In what order? A reducer answers that **deterministically**.

15. Name four reducer patterns and the goal each achieves.

Pattern	Goal
List append / Set union	Keep all values
Last write wins	Keep only one authoritative value
Dict merge	Combine partial structured data

Pattern	Goal
Sum / Max / Min	Aggregate metrics

16. Difference between behavior with and without `add_messages` reducer on the `messages` key?

- Without reducer: `messages` becomes `[new_message]` → previous messages are lost (overwritten).
- With `add_messages` reducer: `new_message` is appended to the existing list → conversation history is preserved. ✓

17. Why is `add_messages` "not just append"? Give the second behavior.

Besides merging two lists of messages, `add_messages` also updates existing messages by ID. If a new message has the **same ID** as an existing message, the new one **replaces** (updates) the old one instead of being appended.

Example: Sending `HumanMessage("Hello again", id="1")` to a state already containing `HumanMessage("Hello", id="1")` will **replace** the original — final state has only `[HumanMessage("Hello again", id="1")]`.

18. What is `MessagesState` and what problem does it solve?

`MessagesState` is a **pre-built conversation-oriented state schema** designed to eliminate boilerplate when your graph state is basically "a growing list of messages." Under the hood, it uses the `add_messages` reducer to merge message lists and update by ID, making the state **append-only by default**.

19. What does `tools_condition` do? When does it return `"tools"` vs `"__end__"`?

`tools_condition` is a **prebuilt routing (conditional-edge) function** used in tool-calling agent loops (ReAct-style). It looks at the **last AI message** in state:

- Returns `"tools"` when the last message is an `AIMessage` and contains one or more `tool_calls`.
- Returns `"__end__"` when there are no tool calls (agent is ready to respond, workflow terminates).

20. Describe the structure of an Agent in LangGraph.

- **Assistant Node** contains the **LLM with Tools**.
- Based on invocation, **if there is a tool call**, the tool is invoked, and the response is sent to a **conditional edge**.
- The conditional edge contains **tools_condition**:
 - If response indicates tools → flow goes back to **assistant node**.
 - Otherwise → graph **ends**.



21. What is a subgraph? Why is this even possible?

A **subgraph** is another graph used as a node inside a larger LangGraph.

It's possible because **nodes in LangGraph are just functions or components** that take state, do work, and return updated state. Since a **compiled graph also behaves like a function** that takes state and returns state, it can act as a node in another graph — creating a **hierarchical workflow**.

22. Name the two subgraph patterns and when to use each.

Pattern	When to Use
Call a subgraph inside a node	Parent and subgraph have different state schemas (no shared keys), or you need to transform state between them.
Add a subgraph as a node	Parent and subgraph share state keys — the subgraph reads from / writes to the same channels as the parent.

23. What is persistence in LangGraph and what is a checkpointer?

Persistence is LangGraph's built-in layer that **saves graph state as checkpoints**.

A **checkpointer** is a component that, when you compile a graph with it, **saves a snapshot of the graph state at every step of execution**, organized into **threads**.

- **Super-step:** Each sequential node is a separate super-step; **parallel nodes share the same super-step.**
 - **Checkpoint:** The state and relevant metadata **packaged at every super-step.**
 - **Thread:** A **collection of checkpoints** representing one workflow run.
 - **StateSnapshot:** The Python type used for checkpoints.
-

25. What is durable execution? List the three steps to enable it.

Durable execution is a technique where a process/workflow **saves its progress at key points**, allowing it to **pause and resume exactly where it left off.**

Steps to enable:

1. **Specify a checkpointer** to save workflow progress.
 2. **Specify a thread identifier** when executing → tracks the execution history for that workflow instance.
 3. **Wrap non-deterministic operations** (random number generation) **or operations with side effects** (file writes, API calls) **inside `@task`** so they aren't repeated on resume.
-

26. Why must non-deterministic operations be wrapped in `@task`?

Because when a workflow resumes after a crash or pause, it **replays previous steps**. If a non-deterministic operation (like `random()`) or a side-effect operation (like a paid API call, file write) **isn't a task**, it would be **executed twice** — leading to duplicate side effects, double charges, or different random values. The `@task` decorator ensures these operations **are not re-executed** on resume.

27. What is LangSmith Deployment? Is it tied only to LangGraph?

LangSmith Deployment is a **workflow orchestration runtime purpose-built for agent workloads**. It takes your agent code and exposes it as a **production-ready API service** with built-in **state management, scaling, monitoring, and persistence.**

No, it's NOT tied only to LangGraph. It is **framework-agnostic** — you can deploy agents built with LangGraph or other frameworks.

Entity	Represents
Assistant	A configured agent
Thread	An execution context with shared state
Run	One execution of the agent
Store	Persistent key-value memory

29. What is an Assistant? How is it different from the underlying graph?

An **Assistant** is just an **instance of a graph with a specific configuration** (e.g., prompts, LLM selection, tools).

The difference: the **graph** defines the **core logic/architecture**, while the Assistant defines the **runtime configuration**. Multiple Assistants can reference the **same graph** with different configurations — e.g., the same ReAct graph can power a "Blog Writer" (Anthropic) and a "Tweet Writer" (OpenAI).

30. What is the LangGraph Store, and how is it different from threads/checkpoints?

The **Store** is the **long-term memory layer** — a persistent **key-{value | document} store** that **survives across threads and conversations**.

Difference: **threads/checkpoints are scoped to runs or threads** (short-term, conversation-level memory), while the **Store outlives them** and can be queried across all sessions/users (long-term memory).

31. Why should you NOT configure the store in your graph code when using Agent Server?

Because the **Agent Server manages it for runtime operations**. When deployed, the server **injects the configured store into graph execution, completely decoupled from the code**. Configuring it in code would conflict with the server's runtime injection.

32. Why is including a tenant ID in the namespace a best practice?

Because long-term memory is stored as JSON documents under a **namespace + key** model. Including the tenant ID (e.g., `("tenant_a", "customers", "c42")`) gives:

- 1. **Data isolation** — memories don't leak across users/tenants

2. Cleaner search boundaries
 3. Easier multi-tenant enforcement
-

33. What is `langgraph.json` and what four things does it tell the runtime?

`langgraph.json` is the **configuration file** that defines how a LangGraph application should be built and run by the LangGraph CLI / Agent Server. It tells the runtime:

1. Which graphs exist
 2. How to load them
 3. What dependencies to install
 4. How to configure the server and runtime
-

34. What does the `dependencies` field do?

It tells the LangGraph CLI / Agent Server **what packages or local code it needs to install** so your graph can import and run successfully. When commands like `langgraph dev`, `langgraph build`, or `langgraph up` are run, the CLI reads `langgraph.json` and uses `dependencies` to **prepare the runtime environment** before loading the graph.

Valid entries:

- `"."` → install the current local Python package
 - `"./"` or `"./my_package"` → a local directory containing `pyproject.toml`/`setup.py`/`requirements.txt`
 - `"langchain_openai"` → a Python package name to install from the package index
-

35. Differentiate the five LangGraph CLI commands.

Command	Purpose
<code>langgraph dev</code>	Starts a lightweight local dev server , ideal for rapid testing.
<code>langgraph build</code>	Builds a Docker image of your LangGraph API server for deployment.
<code>langgraph deploy</code>	Builds and deploys a LangGraph image directly to LangSmith Deployments in a single step.
<code>langgraph dockerfile</code>	Emits a Dockerfile derived from your config for custom builds .

Command	Purpose
<code>langgraph up</code>	Starts the LangGraph API server locally in Docker . Requires Docker running, LangSmith API key for local dev, license for production.



TECHNICAL ANSWERS

1. Build a basic graph with three nodes.

```
python

# Build graph
builder = StateGraph(State)
builder.add_node("node_1", node_1)
builder.add_node("node_2", node_2)
builder.add_node("node_3", node_3)

# Logic
builder.add_edge(START, "node_1")
builder.add_conditional_edges("node_1", decide_mood)
builder.add_edge("node_2", END)
builder.add_edge("node_3", END)

graph = builder.compile()
display(Image(graph.get_graph().draw_mermaid_png()))
```

2. Conditional edge function `decide_mood`.

```
python
```

```

from typing import Literal
import random

def decide_mood(state) -> Literal["node_2", "node_3"]:
    # Often, we will use state to decide on the next node to visit
    user_input = state['graph_state']

    # Here, let's just do a 50 / 50 split between nodes 2, 3
    if random.random() < 0.5:
        # 50% of the time, we return Node 2
        return "node_2"

    # 50% of the time, we return Node 3
    return "node_3"

```

3. Define `MessagesState` using `Annotated`, `add_messages`, `list[AnyMessage]`.

```

python

from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph.message import add_messages
from langchain_core.messages import AnyMessage

class MessagesState(TypedDict):
    messages: Annotated[list[AnyMessage], add_messages]

```

4. Output of `add_messages` with same ID?

```

python

msgs1 = [HumanMessage(content="Hello", id="1")]
msgs2 = [HumanMessage(content="Hello again", id="1")]
add_messages(msgs1, msgs2)

```

Output:

```

python

[HumanMessage(content='Hello again', id='1')]

```

Explanation: Because both messages share the **same ID** `"1"`, `add_messages` **updates the existing message** rather than appending. The final list contains only one message — the

updated one.

5. Invoke a subgraph inside a node (different schemas).

python

```
from typing_extensions import TypedDict

# Subgraph state
class SubgraphState(TypedDict):
    bar: str

# Subgraph
def subgraph_node_1(state: SubgraphState):
    return {"bar": "hi! " + state["bar"]}

# Parent graph state
class State(TypedDict):
    foo: str

def call_subgraph(state: State):
    # Transform parent state → subgraph state
    subgraph_output = subgraph.invoke({"bar": state["foo"]})
    # Transform response → parent state
    return {"foo": subgraph_output["bar"]}
```

6. Add compiled subgraph as a node (shared schema).

python

```
sub_builder = StateGraph(State)

def node1(state):
    ...

def node2(state):
    ...

sub_builder.add_node("node1", node1)
sub_builder.add_node("node2", node2)
sub_builder.add_edge(START, "node1")
sub_builder.add_edge("node1", "node2")
subgraph = sub_builder.compile()

main_builder = StateGraph(State)
main_builder.add_node("analysis", subgraph) # Subgraph plugged in directly
main_builder.add_edge(START, "analysis")
main_graph = main_builder.compile()
```

7. `@task`-decorated function `_make_request`.

```
python

from langgraph.func import task
import requests

@task
def _make_request(url: str):
    """Make a request."""
    return requests.get(url).text[:100]
```

8. Create an assistant named "Blog Writer".

```
python
```

```
assistant = await client.assistants.create(
    "writing_agent",
    context={"model_name": "openai"},
    name="Blog Writer"
)

result = await client.runs.wait(
    thread["thread_id"],
    assistant["assistant_id"],
    input={
        "messages": [...]
    },
)
```

9. Put a profile document for `user123` into the store.

```
python

namespace = ["users", "user123"]
key = "profile"

await client.store.put_item(
    namespace,
    key,
    {
        "name": "Ahmed",
        "preferences": {
            "tone": "concise",
            "language": "en"
        }
    }
)
```

10. Search the store under `["users"]` namespace prefix.

```
python
```

```
results = await client.store.search_items(
    ["users"],
    filter={"preferences.language": "en"},
    query="concise tone"
)
print("SEARCH:", results)
```

11. Sample `langgraph.json`.

```
json
{
  "dependencies": ["./", "langchain_openai"],
  "graphs": {
    "research": "./my_agent/research.py:graph",
    "sql": "./my_agent/sql.py:graph",
    "writer": "./my_agent/writer.py:graph",
    "supervisor": "./my_agent/supervisor.py:graph"
  },
  "env": "./.env"
}
```

12. CLI commands.

- (a) Start a local dev server: `langgraph dev`
- (b) Build a Docker image: `langgraph build`
- (c) Deploy to LangSmith: `langgraph deploy`

COMPLEX / CONCEPTUAL ANSWERS

1. Two parallel branches updating the same `messages` key.

Without a reducer: There's ambiguity — should the final state contain only branch A's update, only branch B's, or both? In what order? The state would likely just be **overwritten**, losing data unpredictably.

With `add_messages`: Both lists are **merged** deterministically via concatenation, with messages sharing IDs being updated. So:

```
new_messages = old_messages + branch_A_messages + branch_B_messages
```

Both branches' contributions are preserved. **Reducers make parallel updates safe and deterministic.**

2. Trade-off: subgraph inside a node vs. as a node.

	Inside a node	As a node
State schema	Different from parent	Shares keys with parent
Effort	Higher — must write a wrapper to map state in/out	Lower — just <code>add_node(compiled_subgraph)</code>
Use case	Privacy, isolation (e.g., per-agent message history)	Tight integration (e.g., shared <code>messages</code> channel)

Inside a node is preferred when each agent should have a **private message history** (multi-agent system). **As a node** is preferred when agents need to **share the same messages channel** for direct coordination.

3. Multi-agent system with private message histories — which subgraph pattern?

Use "Call a subgraph inside a node" (the wrapper pattern).

Why: Each agent needs a **private message history**, meaning the subgraph state schema is **different** from the parent's. The wrapper function maps parent state → subgraph state and back, isolating each agent's conversation while still letting the parent coordinator see only the relevant outputs.

4. Why are reducers crucial for parallelism but not for sequential graphs?

In a **sequential** graph, only **one node updates the state at a time**, so there's no conflict — the second update simply happens after the first.

In a **parallel** graph, multiple nodes update the **same key simultaneously**, creating ambiguity over **what to keep, in what order, and how to combine them**. Reducers resolve this **deterministically**, which is why they're crucial for parallelism but optional in purely sequential flows.

5. Durable execution + `@task` preventing duplicate paid API calls.

When the workflow crashes mid-execution and resumes:

1. The **checkpoint**er has saved the state at every super-step.
2. On resume, LangGraph **replays** the workflow from the last checkpoint.
3. **Without** `@task`, the API call would re-run and you'd be charged twice.
4. **With** `@task`, LangGraph recognizes the operation already completed (the result was checkpointed), so it **skips re-execution** and uses the stored result.

This makes side-effecting operations **idempotent across resumes**.

6. Threads vs. Store — which to use for what?

- **Thread** = short-term, **scoped to a single run/conversation**.
- **Store** = long-term, **persists across threads and conversations**.

(a) **Chat history of a single conversation** → **Thread** (short-term, lives within one run). (b) **User's long-term preferences across all conversations** → **Store** (persists across all threads/sessions; perfect for user profiles).

7. Why is hardcoding Postgres in 10 agents bad vs. relying on Agent Server's injected store?

Problems with hardcoding:

- **Tight coupling** — each agent has its own DB config; changing the DB requires editing 10 codebases.
- **Conflicts at runtime** — Agent Server **injects its own store** at runtime, which would conflict with hardcoded one.
- **No multi-tenant isolation** — harder to enforce namespace-based isolation.
- **Breaks portability** — agents can't be redeployed to different environments without code changes.

With Agent Server's injected store:

- The server **manages the Store** centrally, decoupled from code.
 - Storage backend (Postgres or custom) can be swapped without touching agent code.
 - Multi-tenant namespaces enforce data isolation automatically.
-

8. Same ReAct graph for Blog Writer (Anthropic) and Tweet Writer (OpenAI).

You define **one graph** (the ReAct architecture) and create **two Assistants** that reference it with **different configurations**:

Why this works: Assistants **manage configuration separately from graph logic**, so multiple specialized versions of the same graph architecture can exist with **different runtime behavior** — without duplicating code. Each Assistant can also have its own versions (v1, v2, v3, ...).

9. Why a graph (DAG) over a linear chain?

Two real benefits:

1. **Branching & loops** — A graph supports conditional routing (assistant ↔ tools loop in a ReAct agent), parallel execution, and multiple paths. A chain can't loop or branch dynamically.
 2. **Stateful coordination** — A graph with central state allows **snapshotting, pause/resume, error recovery, and human-in-the-loop**, which a linear chain (with no clear coordination layer) can't provide cleanly.
-

10. Critique: `messages: list[AnyMessage]` (no Annotated, no reducer).

What's wrong: Without `Annotated` and `add_messages`, every update to `messages` **overwrites** the previous list rather than appending. So whenever a node returns `{"messages": [new_message]}`, the state becomes just `[new_message]` — **all conversation history is wiped**.

Fix: Add the `add_messages` reducer via `Annotated`:

```
python
```

```
from typing import Annotated
from langgraph.graph.message import add_messages

class State(TypedDict):
    messages: Annotated[list[AnyMessage], add_messages]
```

Now new messages are appended, and conversation history is preserved.

11. End-to-end: `langgraph dev`.

1. **Reads** `langgraph.json` in the current directory — this tells the CLI which graphs exist, what dependencies to install, and where the `.env` file is.
2. **Installs dependencies** listed in `dependencies` (local project, packages like `langchain_openai`).
3. **Loads the graphs** from the paths specified in the `graphs` block (e.g., `./my_agent/research.py:graph`).
4. **Starts an in-memory local Agent Server**, exposing:
 - **API:** `http://127.0.0.1:2024`
 - **Studio UI:** `https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024`
 - **API Docs:** `http://127.0.0.1:2024/docs`

Why NOT for production: It is **in-memory only** — no real persistence, no scaling, no production-grade infrastructure. The official message says: *"This in-memory server is designed for development and testing. For production use, please use LangSmith Deployment."*

12. Why must conditional edge functions return a string matching a node name?

Because LangGraph uses the returned string as the **routing key** to look up the next node in its registered nodes. When you call `add_node("node_2", ...)`, the name `"node_2"` is registered as a destination. The conditional edge function returns this name, and LangGraph **dispatches the flow** to that registered node.

If the returned string doesn't match a node name, the graph won't know where to route, leading to a runtime error. The `Literal["node_2", "node_3"]` return type hint helps catch this **at type-check time**.

13. Why is `tools_condition` a "routing function"?

Unlike a generic conditional that just returns True/False, `tools_condition` returns the **name of the next node** (`"tools"` or `"__end__"`). It actively **routes** flow based on the LLM's last message:

- If the AI message has `tool_calls` → route to **tools** node.
- Otherwise → route to `__end__`.

Integration with ReAct loop:

```
START → assistant → tools_condition →  
    → tools (if tool_calls) → assistant (loop back)  
    → __end__ (if no tool_calls)
```

This creates the assistant ↔ tools loop characteristic of ReAct agents — the agent keeps calling tools until it has enough information to respond, then exits.

14. Checkpointer + Thread ID + Store — full chatbot stack.

- **Checkpointer:** Saves **every step** of the workflow as a checkpoint. Enables **pause/resume**.
- **Thread ID:** Identifies a **specific conversation instance**. All checkpoints for that conversation are grouped under this thread → enables **conversation memory**.
- **Store:** Saves **long-term memory** (user preferences, profile, history) **across all threads** → enables **cross-session user memory**.

Combined:

- The chatbot **remembers the current conversation** via the thread's checkpoints.
 - It can be **paused and resumed** because the checkpointer captures state at every step.
 - It **remembers the user across many separate conversations** via the Store (e.g., the user's preferred language, name, past topics) — even if a new thread is started.
-

15. Critique: 50-node giant graph file. How would subgraphs improve it?

Problems with a single giant graph:

- **Hard to maintain** — 50 nodes in one file is unreadable.
- **Hard to test** — can't isolate parts of the workflow.
- **Team collaboration suffers** — everyone edits the same file → merge conflicts.
- **No reusability** — logic can't be plugged into other graphs.

Refactoring with subgraphs:

- Break it into 4 specialized subgraphs: **Research**, **SQL**, **Writer**, **Supervisor** — each in its own file.
- Each subgraph is **compiled and reused** as a node in the main graph (or invoked inside a wrapper node).
- Define them all in `langgraph.json`:

json

```
"graphs": {  
  "research": "./my_agent/research.py:graph",  
  "sql": "./my_agent/sql.py:graph",  
  "writer": "./my_agent/writer.py:graph",  
  "supervisor": "./my_agent/supervisor.py:graph"  
}
```

Organizational benefit: Different teams can **work on different parts of the graph independently** ("distributing development"). The Research team owns `research.py`, the SQL team owns `sql.py` — no merge conflicts, clearer ownership, and easier independent testing/deployment.

✅ **End of Answer Key.** Pair this with the study notes — read the question, attempt the answer mentally, then check against the key. Best practice for retention before the quiz!